AI23531 – DEEP LEARNING

EX 4 CONVOLUTIONAL NEURAL NETWORK (CNN) FROM SCRATCH AND FILTER VISUALISATION (CIFAR-10)

AIM

To implement a Convolutional Neural Network from scratch to classify images, train it on labelled images, evaluate its performance and visualise the learned filters of the convolution layer.

PROCEDURE

1. Import PyTorch, torchvision and Matplotlib.
2. Load CIFAR-10 train and test sets, convert to tensors and normalise.
3. Define SimpleCNN with two convolution layers (16 and 32 filters, 3×3), max-pooling and two fully connected layers.
4. Train with Adam (lr = 0.001) and cross-entropy loss for 10 epochs.
5. Evaluate the accuracy on the test set.
6. Normalise and plot the first eight filters of the first convolution layer.

ALGORITHM

1. Start.
2. Load CIFAR-10 and create train/test DataLoaders (batch size 64).
3. Define the CNN: Conv(3→16) → ReLU → MaxPool → Conv(16→32) → ReLU → MaxPool → Flatten → FC(2048→128) → ReLU → FC(128→10).
4. For each epoch and mini-batch: forward pass, compute loss, backpropagate, update weights; print the average loss.
5. Run the model on the test set without gradients and calculate accuracy.
6. Take the weights of conv1, scale each filter to 0–1 and display it as an image.
7. Stop.

PROGRAM

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Step 1: Load and preprocess CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True, transform=transform)

trainloader = torch.utils.data.DataLoader(
    trainset, batch_size=64, shuffle=True)
```

```
testset = torchvision.datasets.CIFAR10(
    root='./data', train=False, download=True, transform=transform)

testloader = torch.utils.data.DataLoader(
    testset, batch_size=64, shuffle=False)

classes = trainset.classes

# Step 2: Define CNN
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()

        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)

        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))

        x = x.view(-1, 32 * 8 * 8)

        x = self.relu(self.fc1(x))
        x = self.fc2(x)

        return x

# Step 3: Train CNN
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = SimpleCNN().to(device)

lossfn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    running_loss = 0.0

    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()

        outputs = model(inputs)
        loss = lossfn(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: "
          f"{running_loss / len(trainloader):.4f}")

# Step 4: Evaluate model
correct = 0
total = 0
```

```
with torch.no_grad():
    for images, labels in testloader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        _, predicted = outputs.max(1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Accuracy on test data: "
      f"{100 * correct / total:.2f}%")

# Step 5: Visualize learned filters
def visualize_filters(layer, n_filters=8):
    filters = layer.weight.data.cpu()

    fig, axs = plt.subplots(1, n_filters, figsize=(15, 4))

    for i in range(n_filters):
        f = filters[i]
        f = (f - f.min()) / (f.max() - f.min())

        axs[i].imshow(f.permute(1, 2, 0))
        axs[i].axis('off')
        axs[i].set_title(f'Filter {i+1}')

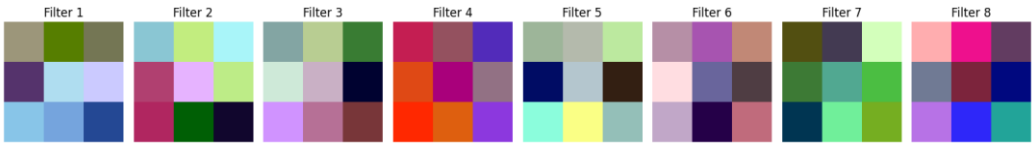
    plt.tight_layout()
    plt.show()

visualize_filters(model.conv1)
```

OUTPUT

```
Epoch 1, Loss: 1.4459
Epoch 2, Loss: 1.1117
Epoch 3, Loss: 0.9728
Epoch 4, Loss: 0.8715
Epoch 5, Loss: 0.7943
Epoch 6, Loss: 0.7228
Epoch 7, Loss: 0.6605
Epoch 8, Loss: 0.5987
Epoch 9, Loss: 0.5430
Epoch 10, Loss: 0.4916
Accuracy on test data: 68.90%
```

Console output



Learned filters of the first convolution layer

RESULT

The training loss fell from 1.4459 to 0.4916 over 10 epochs and the CNN achieved 68.90% accuracy on the CIFAR-10 test data. The learned filters were visualised.